

Entrega final:

Optimización Estocástica del Flujo e Inventario para una Gasolinera

Diego Alejandro Salinas Ruiz A01286252
Juan Eduardo Gonzalez Macias A01285265
Mariana Villarreal R. Vela A01722821
Ximena Padrón Contreras A00840445
Jorge Luis Muñoz Reyes A00838647
Hector Enrique Campbell Salas A00839531
Bernardo Lugo Albor A01707191
José Alfredo Almaguer Cruz A00839015

Diciembre 2025

1 Resumen

En este trabajo se realiza una amplia investigación y análisis de la problemática para desarrollar métodos que optimicen y faciliten las operaciones de una empresa. En colaboración con PhaiMat Labs, se trabajará con una gasolinera con cinco estaciones operando. Esta empresa solicita mejorar sus ganancias volviendo más eficiente y optimizando tres áreas de trabajo: predicción de la demanda de su servicio, eficientizar las cadenas de suministro y mantener un flujo de caja constante; todo esto minimizando perdidas económicas importantes dado que es un mercado volátil. Para plantear una demanda estocástica se emplea un método predictivo SARIMA usando datos históricos y manteniendo un intervalo de confianza en los resultados. Después se modela un sistema de optimización para maximizar el tiempo entre cada reabastecimiento del combustible en cada estación tomando en cuenta la demanda de cada una. Usando estos datos, se implementó un modelo multi-etapa para imitar decisiones optimas adaptadas a los resultados esperados. Por último, se analizan los resultados en conjunto para unificar las simulaciones en un modelo que permita cumplir los objetivos de optimización y minimizar los costos internos de la empresa.

2 Introducción

Phaimat Labs es una consultora creada con la convicción de ofrecer soluciones reales a las empresas integrando rigor teórico, aplicación práctica y desarrollo tecnológico innovador. Particularmente, en esta ocasión, una gasolinera solicitó de los servicios de dicha consultora, con la intención de optimizar sus procesos y como consecuencia maximizar sus ganancias, es por ello que en este trabajo recabamos una investigación con la finalidad de proporcionar un servicio adecuado en conjunto a Phaimat Labs para que la gasolinera pueda beneficiarse de un buen análisis de datos que les permita tomar la mejor decisión posible en concordancia con lo que buscan.

En base a la extensión del equipo e identificando las principales áreas de relevancia para la comprensión exhaustiva de la problemática que presenta la optimización de estaciones de gasolina y sus ganancias es que decidimos realizar el trabajo consistiendo de tres principales etapas que nos permitan:

- Modelar la demanda temporal de consumo de gasolina con un modelo predictivo estocástico
- Modelar las cadenas de suministro para las estaciones de gasolina para optimizar el tiempo de reabastecimiento
- Optimizar el flujo de caja para minimizar las pérdidas y mantener un flujo de efectivo constante

3 Metodología

3.1 Modelo de la demanda estocástica

Basandose en datos sobre las ventas de las cinco estaciones de servicio de combustible, se propondrá un modelo estocástico que simulará una predicción de consumo de este servicio para un periodo de tiempo determinado. Se utilizó una base de datos históricos: [1, 2]

3.1.1 Planteamiento del modelo y SARIMA

Se definen los parámetros y métricas del modelo de predicción de demanda de gasolina. La demanda diaria se modela inicialmente como un proceso estocástico de Ornstein–Uhlenbeck (OU), ya que cumple:

- Regresión a la media: el proceso revierte hacia la media μ con velocidad θ .
- Proceso gaussiano: cualquier colección finita $(X_{t_1}, \dots, X_{t_n})$ tiene distribución normal multivariada.
- Propiedad de Markov: el futuro sólo depende del valor actual.
- Distribución estacionaria: para tiempos grandes, el proceso converge en distribución a una normal.

Se considera la serie de tiempo del promedio consumido de gasolina por día,

$$X_n := X_{t_n},$$

y por el Teorema del Límite Central se concluye que estos promedios siguen aproximadamente una distribución normal.

A partir de datos de consumo de combustible en fuentes de gobierno, se observa que la demanda es autorregresiva y tiende a regresar a una media de largo plazo:

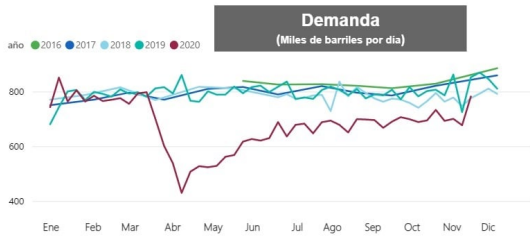


Figure 1: Demanda de gasolina anual (SENER)

Esta gráfica ilustra el comportamiento de largo plazo, pero el modelo de interés se enfoca en un horizonte más corto para optimizar el abastecimiento de gasolineras en un periodo definido. Además, la gráfica anterior corresponde a la demanda nacional; se requiere especializar el análisis a la región Noroeste:

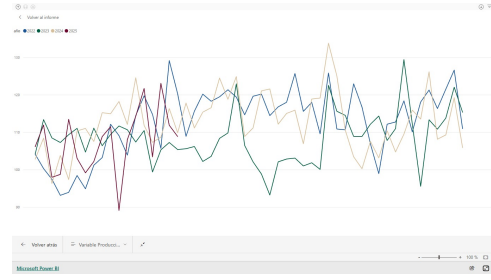


Figure 2: Comparación de la demanda de la gasolina en la región Noroeste

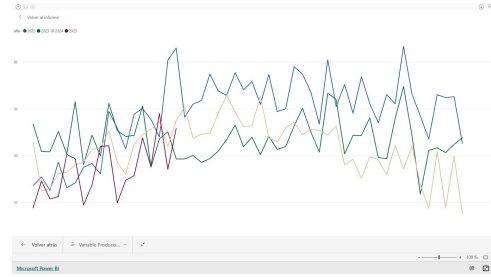


Figure 3: Comparación de la demanda de DIESEL en la región Noroeste

Cualitativamente se aprecia la existencia de temporadas de alto y bajo consumo. Al enfocar los datos en Nuevo León se observa aún más claro este patrón:

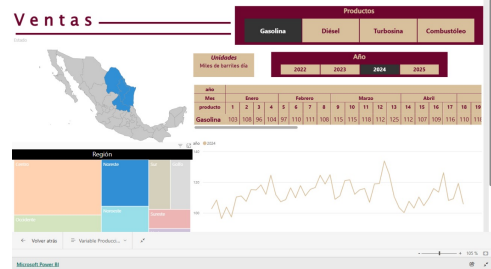


Figure 4: Demanda de miles de barriles por día en Nuevo León (SENER)

En este caso, la media cambia con los meses, por lo que un OU estrictamente estacionario no es adecuado para predicciones en un horizonte corto. Esto motiva el uso de un modelo SARIMA, que extiende la idea autorregresiva del OU pero incorpora tendencias y estacionalidad explícita. [3]

Para obtener un modelo más flexible se recurre a los modelos ARIMA, que permiten incluir términos autorregresivos (AR), de medias móviles (MA) e integrar (diferenciar) la serie cuando la media o la varianza no son estacionarias. Si una serie es integrada de orden 1 (I(1)), sus primeras diferencias son estacionarias (I(0)).

Decimos entonces que la serie sigue un modelo ARIMA(p, d, q) Sdamodar, donde p es el número de términos autorregresivos, d el orden de integración y q

el número de términos de medias móviles. Según los valores de estos parámetros se obtienen modelos AR, ARMA o ARIMA.

Finalmente, se incorpora el componente estacional, ya que la serie muestra patrones que se repiten cada cierto número de periodos. El modelo SARIMA se expresa como:

$$\text{SARIMA}(p, d, q)(P, D, Q)_s$$

donde:

- p, d, q : partes no estacionales.
- P, D, Q : partes estacionales:
 - P : término autorregresivo estacional (usa $X_{t-s}, X_{t-2s}, \dots$)
 - D : diferencia estacional ($X_t - X_{t-s}$)
 - Q : término de medias móviles estacional (errores en $t - s, t - 2s, \dots$)
- s : periodo estacional (por ejemplo, 12, 4, 7, 24, etc.).

3.1.2 Implementación del modelo SARIMA y selección de parámetros

A partir del análisis previo se ajustó un modelo $\text{SARIMA}(p, d, q)(P, D, Q)_s$ para describir la evolución temporal del consumo de combustible. El modelo incluye coeficientes asociados a:

- **Estacionalidad** (parámetros P, D, Q), que capturan patrones que se repiten cada s periodos.
- **Componente autorregresiva** (AR), que relaciona la demanda actual con valores pasados.
- **Componente integral** (diferencias d y D), utilizada para eliminar tendencias y lograr estacionariedad.
- **Medias móviles** (MA), que modelan el efecto de choques o errores pasados sobre la demanda.

Los parámetros del modelo se estiman mediante máxima verosimilitud. En la salida del ajuste se reportan, para cada coeficiente, su valor estimado (*coef*), el error estándar (*std err*), el estadístico z y el valor p , que indican la significancia estadística de cada término en la explicación de la serie.

La ecuación que gobierna al modelo es la siguiente:

$$\Phi_P(L^s) \phi_p(L) (1 - L)^d (1 - L^s)^D y_t = \Theta_Q(L^s) \theta_q(L) \varepsilon_t$$

donde

$$\phi_p(L) = 1 - \phi_1 L - \phi_2 L^2 - \dots - \phi_p L^p,$$

$$\Phi_P(L^s) = 1 - \Phi_1 L^s - \Phi_2 L^{2s} - \dots - \Phi_P L^{Ps},$$

$$\theta_q(L) = 1 + \theta_1 L + \theta_2 L^2 + \dots + \theta_q L^q,$$

$$\Theta_Q(L^s) = 1 + \Theta_1 L^s + \Theta_2 L^{2s} + \dots + \Theta_Q L^{Qs}.$$

El operador de rezago L se define como $L^k y_t = y_{t-k}$, lo cual permite expresar de forma compacta las dependencias temporales del proceso.

El modelo SARIMA combina de manera multiplicativa los componentes no estacionales y estacionales. Los términos AR y MA no estacionales, representados por los polinomios $\phi_p(L)$ y $\theta_q(L)$, capturan dependencias de corto plazo. Por otro lado, los polinomios estacionales $\Phi_P(L^s)$ y $\Theta_Q(L^s)$ modelan patrones que se repiten cada s periodos. Las diferenciaciones $(1 - L)^d$ y $(1 - L^s)^D$ eliminan tendencia y estacionalidad, respectivamente, diferencian para encontrar un proceso estacionario.

La estructura multiplicativa del modelo implica que, al expandir los polinomios, pueden aparecer términos AR y MA combinados en rezagos como L^{p+s} , resultado de la interacción entre componentes estacionales y no estacionales.

Finalmente, el término ε_t corresponde a un proceso de ruido blanco con media cero, varianza constante y sin autocorrelación. El parámetro s denota la periodicidad estacional: por ejemplo, $s = 12$ en datos mensuales con estacionalidad anual.

Criterio de información AIC

Para seleccionar la mejor combinación de órdenes (p, d, q, P, D, Q, s) se probaron varios modelos candidatos y se compararon mediante el Criterio de Información de Akaike (AIC), definido como:

$$AIC = -2 \ln(L) + 2k,$$

donde L es la máxima verosimilitud del modelo y k el número de parámetros estimados. Un AIC más bajo indica un mejor compromiso entre: (i) buen ajuste a los datos (mayor verosimilitud) y (ii) parsimonia (menor número de parámetros). El modelo utilizado en este trabajo es el que obtuvo el menor valor de AIC entre todas las especificaciones candidatas. [4]

Pronóstico de precios y ruido gaussiano

Una vez fijados los coeficientes del modelo, se generan pronósticos para el horizonte de interés. La Figura 5 muestra el ajuste histórico y la proyección semanal del precio de la gasolina en México, así como el intervalo de confianza al 95% que resulta de la varianza de los errores del modelo.



Figure 5: Precio de gasolina en México: histórico y pronóstico semanal con intervalo de confianza al 95 % (modelo SARIMA).

Para capturar las fluctuaciones aleatorias alrededor del pronóstico medio, se incorpora un término de ruido gaussiano en la parte predictiva. Se supone que los errores siguen una distribución normal $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$, donde σ^2 se estima a partir de la variabilidad histórica. De este modo, el valor pronosticado en cada periodo se expresa como:

$$\hat{X}_t = \mu_t + \varepsilon_t,$$

donde μ_t es la trayectoria media generada por el modelo SARIMA y ε_t representa las variaciones aleatorias alrededor de esa media. Los intervalos de confianza se construyen a partir de esta distribución, cubriendo aproximadamente el 95 % de los posibles escenarios.

Predicción de la demanda por estación

Con el modelo calibrado se realiza la proyección de la demanda diaria por estación. La figura 6 presenta el caso de una estación representativa, donde se observa la serie histórica, el pronóstico obtenido con el modelo y las bandas de confianza asociadas al ruido gaussiano:

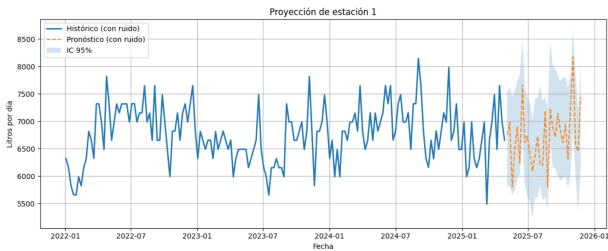


Figure 6: Proyección de la demanda diaria para una estación: histórico, pronóstico y banda de confianza al 95 %.

Estas proyecciones permiten cuantificar la demanda esperada y la incertidumbre asociada, lo cual es esencial para dimensionar el inventario y optimizar el abastecimiento de las estaciones considerando la naturaleza estocástica de la demanda.

3.2 Optimización temporal de la cadena de suministro

El planteamiento de este problema consiste en proponer un sistema de gestión de inventarios para cinco estaciones de servicio de combustible.

El objetivo es maximizar el tiempo entre reabastecimiento consecutivos, lo que reduce costos operativos y mejora la eficiencia logística. Se buscará determinar cuánto combustible debe entregarse a cada estación durante un reabastecimiento, de manera que se maximice el tiempo hasta el siguiente reabastecimiento necesario. [5]

Este planteamiento busca coordinar el tiempo máximo de reabastecimiento de las cinco estaciones sujeto a restricciones operativas. El consumo de cada estación depende de su demanda individual basada en un pronóstico SARIMA, por lo que está escalado de manera temporal.

3.2.1 Formulación y planteamiento matemático

Se utiliza un plantemiento matemático típico de problemas de optimización de cadenas de suministro, utilizando restricciones de límites de inventario [6].

Se tienen dos variables de decisión

- $Q_i \leq 0$: Cantidad a entregar a la estación i en litros
- $T \geq 0$: Tiempo hasta el siguiente reabastecimiento en semanas

Y las siguientes restricciones

- **Restricción de inventario mínimo:** Para cada estación I_i donde $i = 1, \dots, 5$:

$$I_i^{actual} + Q_i - \mu_i \cdot T \geq I_{min,i}$$

Esta restricción asegura que, después de T semanas de consumo a la tasa promedio μ_i , el inventario no caiga por debajo de un mínimo establecido.

- **Restricción de capacidad máxima de almacenamiento:** Para cada estación I_i :

$$I_i^{actual} + Q_i \leq I_{max,i}$$

- **Restricción de capacidad del vehículo:** El total de combustible distribuido no puede exceder la capacidad del medio de transporte, en este caso 40,000 litros.

$$\sum_{i=1}^5 Q_i \leq C = 40,000$$

[7]

3.2.2 Optimización lineal

Se planteó este problema como uno de programación lineal (LP), donde nuestra función objetivo es simplemente planteada como,

$$\max(T)$$

haciendo uso de la biblioteca de Python CVXPY para optimización convexa. Se resuelve utilizando ECOS (Embedded Conic Solver), un solucionador bastante eficiente para este tipo de problemas de programación lineal y cónica de segundo orden.

Tomando la demanda estocástica planteada anteriormente en la sección 3.1, la cantidad de combustible que se tiene un tiempo t , o el inventario, se actualiza semanalmente de la siguiente forma.

$$I_i(t+1) = I_i(t) - d_i^{real}(t)$$

donde se tiene la demanda con distribución normal de la forma

$$d_i^{real}(t) \sim \mathcal{N}(\mu_i(t), \sigma_i(t))$$

con media $\mu(t)$ y desviación estándar $\sigma(t)$, para cada estación.

El modelo considera inventarios mínimos y máximos para cada estación

$$I_{min} = \begin{bmatrix} 10,000 \\ 12,000 \\ 9,000 \\ 11,000 \\ 10,000 \end{bmatrix} \text{ litros}, \quad I_{max} = \begin{bmatrix} 68,991.05 \\ 55,981.45 \\ 49,611.06 \\ 24,046.68 \\ 51,829.93 \end{bmatrix} \text{ litros}$$

Y se comienza la simulación con un inventario de $I_0 = 20,000$ litros.

Política de reabastecimiento:

- El sistema de reabastecimiento se activa cuando al menos una estación alcanza su inventario mínimo.
- Se resuelve el problema LP para determinar los valores de Q_i^* y T_i^* de cada estación
- se aplica el reabastecimiento $I_i \rightarrow I_i + Q_i$ para toda i
- Se continúa la simulación

Las ventajas de este enfoque son la garantía en la solución optima global para el problema formulado y que se pueden manejar más estaciones o restricciones adicionales. Este método permite una manera facil y eficiente de modelar estos objetivos.

Sin embargo, es importante considerar que el modelo tiene algunas limitaciones. Por ejemplo, asume que la demanda promedio μ_i es constante durante T semanas, y no se están tomando en cuenta costos de transporte, almacenamiento u otros costos operativos.

3.3 Maximización del flujo de caja

En el flujo de caja decidimos utilizar el modelo estocástico multi-etapa, esto porque el modelo puede tomar decisiones en diferentes momentos del tiempo ajustándose a los resultados que se le estén dando, imitando las decisiones que se toman en base a lo que se espera para un futuro sin intentar predecir por completo lo que va a pasar. Además, este modelo modela la incertidumbre de las variables en el tiempo, mostrando la incertidumbre de diferentes espacios en el tiempo. El modelo genera decisiones adaptativas, en vez de tener las mismas opciones de decisiones, este evalúa las opciones que tiene en cada instante y ver que es lo que necesita hacer para llegar a una mejor optimización en el flujo de caja. El último beneficio que aprovechamos del modelo es que permite generar restricciones para las variables, así estas pueden ser modificadas para cada estación, por lo que el modelo se ajusta a los límites que se le impongan.[8, 9]

La función estocástica que estaremos aproximando modela el; Flujo total (s):

$$\sum_{t=1}^T \sum_{j \in \mathcal{J}} \left(p_{tjs}^{\text{sell}} x_{tjs} - p_{tjs}^{\text{buy}} y_{tjs} - h_j I_{tjs} - f_{tj} \right), \quad (1)$$

Donde:

- t indexa periodos $t = 1, \dots, T$,
- $j \in \mathcal{J}$ indexa estaciones,
- s denota el escenario,
- x_{tjs} son las ventas (sell), y_{tjs} las compras (buy),
- I_{tjs} el inventario al final del periodo,
- $p_{tjs}^{\text{sell}}, p_{tjs}^{\text{buy}}$ precios en t, j, s ,
- h_j coste de holding por unidad en la estación j ,
- f_{tj} coste fijo en t, j .

Para llegar a esta aproximación lo que decidimos es buscar la esperanza en la incertidumbre para encontrar los posibles escenarios dentro de estas probabilidades y conocer cual es la más probable. El modelo se creó por periodos ya que no es factible modelarlo por día ya que el cambio no puede ser muy brusco, entonces las esperanzas en el flujo por periodo es lo que busca el modelo.

Al modelarlo de esta manera el modelo necesita aplicar la no-anticipatividad por lo que no puede tomar decisiones en base a valores del futuro, por ende, las decisiones se toman en base al valor esperado y no a lo que estará pasando. Esto porque en la vida real no puedes predecir que pasará, solo tomar decisiones en base a las probabilidades que tienes en el momento.[10, 3, 11]

4 Resultados gráficos y numéricos

4.1 Modelo de la demanda estocástica

4.1.1 Predicción del precio de la gasolina en México.

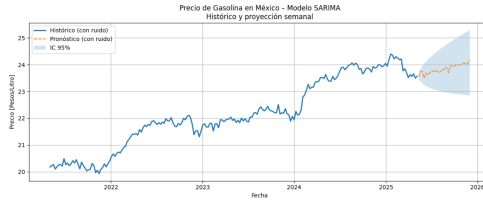


Figure 7: Predicción del precio de la gasolina regular en México.

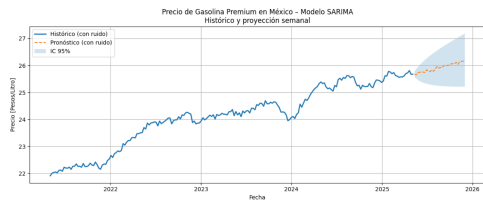


Figure 8: Predicción del precio de la gasolina premium en México.

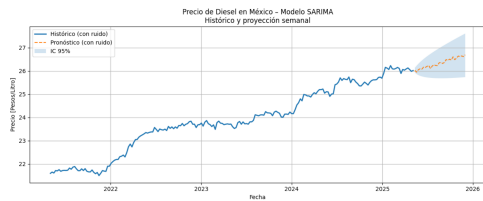


Figure 9: Predicción del precio del Diesel en México.

Las Figuras 7, 8 y 9 muestran la predicción del precio de la gasolina regular, premium y del diésel en México, respectivamente, obtenidas a partir del modelo SARIMA propuesto. En los tres casos se aprecia que el modelo reproduce adecuadamente la tendencia creciente observada en los datos históricos y genera proyecciones suaves y coherentes para el horizonte de pronóstico.

Los intervalos de confianza al 95 % se ensanchan conforme avanza el tiempo, lo cual refleja el incremento natural de la incertidumbre en un modelo estocástico. La banda de predicción permanece, sin embargo, acotada alrededor de la trayectoria esperada, indicando que las fluctuaciones simuladas mediante el ruido gaussiano son consistentes con la variabilidad histórica de cada tipo de combustible.

4.1.2 Validez de la predicción del precio de la gasolina en México.

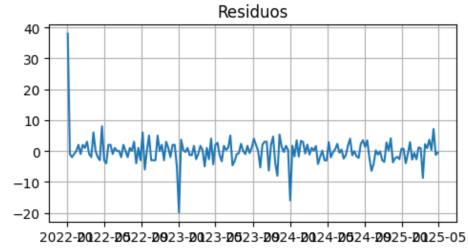


Figure 10: Residuos de la predicción del precio de la gasolina.

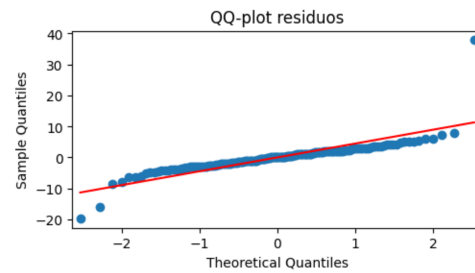


Figure 11: Residuos de la predicción del precio de la gasolina contra una distribución normal.

La validez del modelo SARIMA se evaluó a partir del análisis de los residuos. En la Figura 10 se observa que los residuos de la predicción del precio de la gasolina oscilan alrededor de cero, sin tendencias aparentes ni patrones sistemáticos en el tiempo. Además, su magnitud se mantiene relativamente acotada, lo cual sugiere que el modelo captura adecuadamente la estructura principal de la serie y que la varianza de los errores es aproximadamente constante.

Por otro lado, la Figura 11 presenta el QQ-plot de los residuos frente a una distribución normal teórica. La mayoría de los puntos se alinean cerca de la recta de referencia, indicando que la suposición de normalidad es razonable para efectos de inferencia y construcción de intervalos de confianza, con ligeras desviaciones en las colas. En conjunto, estos resultados respaldan la validez estadística del modelo para el pronóstico del precio de la gasolina en México.

4.1.3 Predicción de demanda de gasolina por estación.

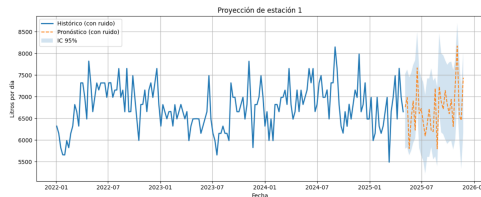


Figure 12: Predicción de la demanda de la estación 1

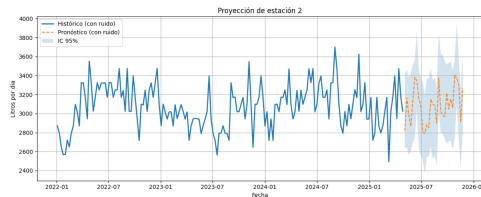


Figure 13: Predicción de la demanda de la estación 2

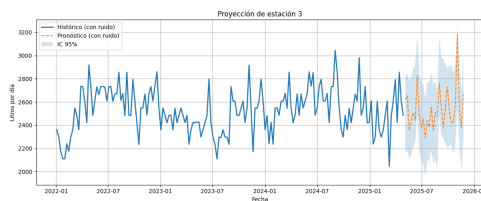


Figure 14: Predicción de la demanda de la estación 3

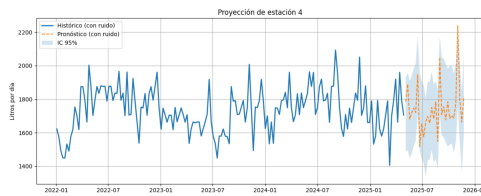


Figure 15: Predicción de la demanda de la estación 4

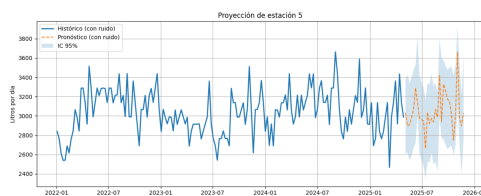


Figure 16: Predicción de la demanda de la estación 5

Las Figuras 12 a 16 presentan la predicción de la demanda diaria de gasolina para las cinco estaciones analizadas. En todos los casos se observa que el modelo reproduce razonablemente la variabilidad histórica de

cada serie y genera pronósticos consistentes con los niveles de consumo recientes de cada estación.

Las trayectorias pronosticadas se acompañan de intervalos de confianza al 95 %, cuya amplitud varía entre estaciones de acuerdo con la dispersión histórica de los datos: estaciones con mayor variabilidad presentan bandas más anchas, mientras que en aquellas con consumo más estable los intervalos son más acotados. Esto permite cuantificar explícitamente la incertidumbre asociada a la demanda futura y utilizar dicha información para planear inventarios y abastecimiento, considerando tanto el escenario esperado como los posibles desvíos en cada estación.

4.2 Optimización temporal de la cadena de suministro

Los resultados de esta etapa del trabajo se pueden observar al graficar la evolución temporal del inventario en cada estación. Se presenta una gráfica que muestra la evolución temporal de los inventarios de todas las estaciones, y posteriormente se presenta la evolución temporal de cada estación, observando cotas de inventario mínimas y máximas.

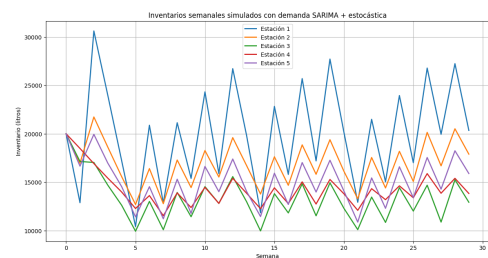


Figure 17: Evolución del inventario - Todas las estaciones

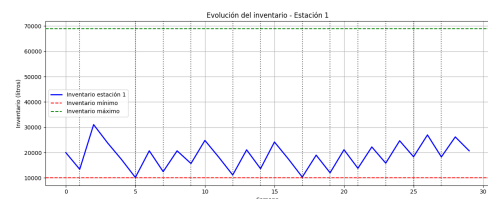


Figure 18: Evolución del inventario - Estación 1

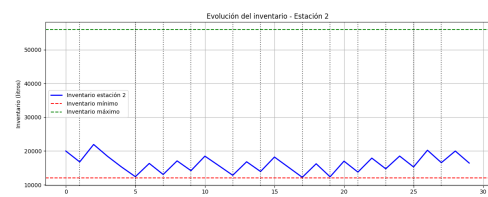


Figure 19: Evolución del inventario - Estación 2

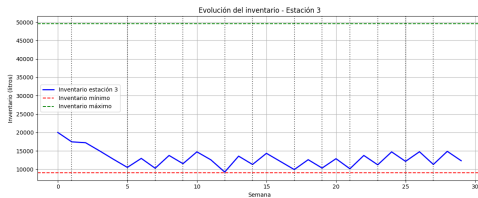


Figure 20: Evolución del inventario - Estación 3

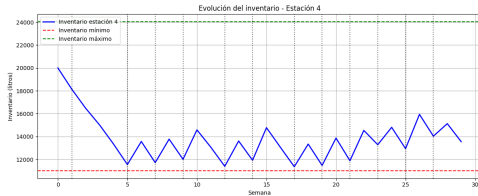


Figure 21: Evolución del inventario - Estación 4

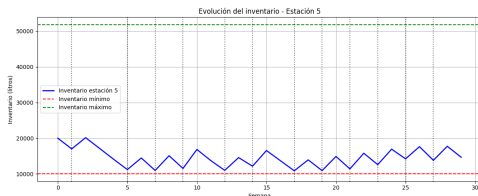


Figure 22: Evolución del inventario - Estación 5

Se puede observar que el reabastecimiento ocurre cuando uno de los inventarios de las estaciones llega al límite inferior, mientras que la cantidad reabastecida por estación proviene de las soluciones obtenidas por medio de programación lineal. Las cantidades reabastecidas por estación, así como los intervalos de reabastecimiento, se observan en la siguiente tabla:

#	Semana	Q1 (Litros)	Q2 (Litros)	Q3 (Litros)	Q4 (Litros)	Q5 (Litros)
	1	23335.58	7850.88	2396.28	0	6417.26
	5	17910.46	6824.12	5946.77	3228.53	6090.12
	7	15172.7	7040.67	6770.63	4399.43	6616.58
	9	15704.9	7040.17	5542.43	3535.64	8176.86
	11	17385.92	6873.5	5367.76	4256.85	6115.98
	14	16128.57	6644.57	6190.23	3783.43	7253.2
	16	16144.81	7063.62	5755.27	3873.7	7162.61
	18	17582.84	6443.36	6068.14	4111.04	5794.63
	21	14887.34	7436.19	5899.94	4057.5	7719.04
	23	15769.76	6917	6394.91	3411.45	7506.88
	25	15162.22	8392.09	4765.74	4507	7172.94
	27	15069.21	6983.3	6734.54	3474.71	7738.24

Figure 23: Tabla de resultados - Reabastecimiento

Se puede observar que el tiempo óptimo promedio de reabastecimiento es de aproximadamente 2 semanas (quincenal), sin embargo, al utilizar las combinaciones de cantidades reabastecidas por estación que aporta el proceso de optimización, se puede llegar a alargar este tiempo de reabastecimiento a 3 semanas.

4.3 Maximización del flujo de caja

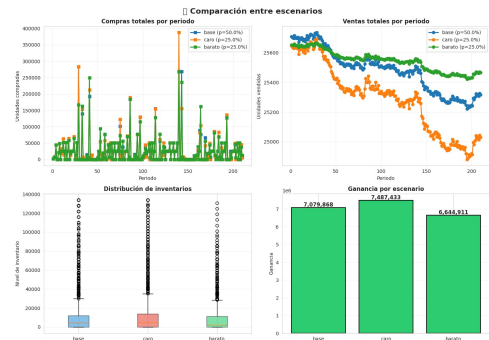


Figure 24: Grafica de los resultados economicos

Como se puede observar en la figura 24 se crearon diferentes escenarios. Uno para los precios base, otro para precios altos y finalmente uno para precios bajos. Estos escenarios modelan como el nombre lo dice lo que estaría pasando en cada escenario con diferentes precios esperados de gasolina. Como no podemos conocer realmente como se comportará este, modelamos diferentes escenarios para ver como es que se podría llegar a comportar el mercado en cada uno de estos. También se observa que en la distribución de inventarios estos valores siempre se mantienen en promedio por debajo de los 10,000 litros. Obteniendo muchos valores atípicos debido a que son los momentos en los cuales se recarga gasolina. Observamos finalmente que las ganancias por escenario son muy parecidas, sin embargo, cuando el costo de gasolina es mayor las ganancias también lo son. Esto nos dice que el volumen de venta no importa tanto si las ganancias por venta son mayores.

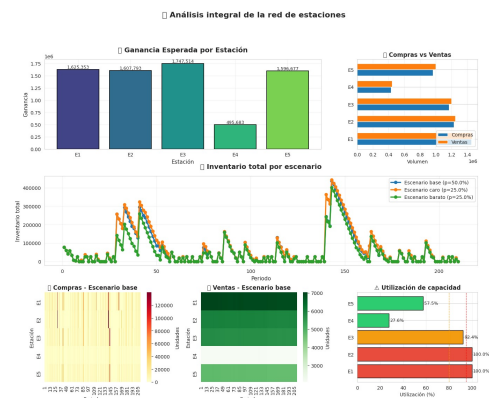


Figure 25: Resultados por estación

Ahora, en la figura 25 observamos las ganancias esperadas por estación, se ve como en la estación N.4 las ventas son mucho menores que las demás. Se necesitaría de un análisis más profundo del porque ocurre esto, aunque algunos factores para esto pueden llegar a ser la localización, percepción de calidad de gasolina

por el cliente, atención al cliente, entre otros factores. Después se observa el inventario total de las estaciones durante el periodo de análisis. Después podemos observar los periodos y la cantidad de compra en cada estación, a un lado están las ventas por estación y finalmente el uso de capacidad por estación.

Por lo que logramos observar y por nuestro modelo podemos suponer que la estación 4 además de tener menos ventas esta se mantiene muy baja todo el tiempo, esto está directamente relacionado entre sí, ya que si no se esperan ventas no es factible comprar demás.

5 Discusión

Los resultados muestran que el proyecto sí logró reflejar la dinámica estocástica de la red gasolinera y logró generar decisiones operativas convenientes y coherentes.

El modelo SARIMA reproduce la estacionalidad observada en los datos reales de consumo y genera pronósticos con residuos aproximadamente normales, lo que indica un ajuste adecuado para alimentar los siguientes módulos. En la optimización de la cadena de suministro, las trayectorias de inventario por estación respetan las cotas mínimas y máximas, y el modelo lineal de reabastecimiento permite operar en promedio, con ciclos quincenales que pueden extenderse hasta tres semanas sin sobrepasar restricciones. Esto reduce la frecuencia de viajes del camión y potencialmente los costos logísticos.

El modelo estocástico multi-etapa de flujo de caja muestra que, bajo distintos escenarios de precio, los inventarios se mantienen en promedio por debajo de 10 000 litros con picos asociados a las recargas, y que el escenario de gasolina cara, aunque reduce el volumen vendido, produce las mayores ganancias debido al mayor margen unitario; además, nos muestra que algunas estaciones concentran la mayor parte de la rentabilidad mientras que otras, como la estación 4, operan con baja utilización y contribuyen poco al resultado global. Aunque existen simplificaciones (por ejemplo, costos de transporte y elección de límites inferiores) los modelos acoplados de demanda, inventarios y flujo de caja proporcionan una base útil para tomar decisiones que mejoren la disponibilidad del producto y la eficiencia financiera de la red.

6 Conclusiones

6.1 Demanda Estocástica

Empleando el modelo SARIMA fue posible obtener una predicción y un intervalo de confianza para la demanda y el precio de la gasolina y el diésel. La principal ventaja que ofrece el modelo propuesto es que este ajusta sus parámetros en base a los datos históricos con los que es alimentado. En este sentido, la propuesta que

planteamos para el socioformador es que realicé un registro histórico de sus ventas de gasolina y ajuste el modelo en base a dichos datos con el objetivo de obtener predicciones de demanda adecuadas a cada estación concreta.

Asimismo, debido a que la información de venta de gasolina proporcionada por el socioformador fue poca, como equipo, la manera en que decidimos abordar esto fue a través de buscar los datos históricos de Nuevo León, modelar la predicción de la demanda y escalar los resultados de acuerdo con las ventas promedio de cada estación. Finalmente, para distinguir entre estaciones, se implementó un ruido gaussiano basado en la varianza que el modelo SARIMA calculó para la demanda. Este ruido, toma en consideración factores estocásticos que intervienen en la demanda de gasolina.

6.2 Cadenas de suministro

Para la optimización de la cadena de suministros, se partió de la demanda obtenida por el modelo SARIMA, lo cual le agrega la faceta estocástica y real al modelo. El método elegido fue programación lineal. Esta elección se debe a la naturaleza lineal y convexa del problema; esto proviene de cómo fueron planteadas la función objetivo y las restricciones.

También nos planteamos utilizar algoritmos genéticos para encontrar las soluciones óptimas, sin embargo concluimos que se llegaría a las mismas soluciones pero con un mayor costo computacional. Sería interesante ver si es adecuado utilizar otros métodos de optimización metaheurísticos (como algoritmos genéticos), al agregarle complejidad al modelo.

Algo que pudimos haber mejorado en el modelo es la definición de los límites inferiores, ya que aunque el modelo facilita cambiarlos según el nivel de seguridad que se requiera, no implementamos directamente un factor que defina los límites inferiores a partir del nivel de seguridad que se solicite. Esto también causó que hubiera estaciones que tenían un *stock* muy superior al límite inferior, lo cual implica pérdidas para la empresa. En general, la solución alcanzada puede ser una buena base para llegar a agregarle complejidad al modelo, llegando a soluciones más específicas para la red gasolinera del socioformador.

6.3 Flujo de caja

Para la parte de flujo de caja, se usó un método que nos permita calcular, por periodos, la diferencia entre ingresos por venta y egresos por compra para obtener el dinero líquido generado por la red de estaciones. Con esto se puede medir la liquidez, la rentabilidad y la viabilidad del proyecto, lo cual nos permite tomar decisiones financieras que beneficien a la empresa. Específicamente, se implementó un esquema estocástico multi-etapa apoyado por redes neuronales para generar esce-

narios realistas y responder simultáneamente al “qué decisiones tomar” y “cómo adaptarlas” para mantener un flujo de caja estable bajo incertidumbre. Por escenario, podemos ver que aunque el escenario caro reduce el volumen vendido respecto al base y al barato, su mayor margen unitario produce la ganancia total más alta. Es así como el flujo de caja se convierte en una herramienta que compara políticas de precio: maximizar ventas no implica maximizar rentabilidad; lo que es importante es el margen y el uso eficiente de la capacidad. Adicionalmente, se puede observar que la distribución de inventarios se mantiene en promedio por debajo del umbral operativo seleccionado, con picos asociados a recargas. Esto nos indica que el modelo logra cierto equilibrio entre disponibilidad y capital inmovilizado. Enfocándonos en las estaciones, los resultados muestran que algunas estaciones operan casi a plena capacidad y concentran gran parte de la ganancia, mientras que otras (como la estación con menores ventas y utilización) aportan poco al resultado global. Con el modelo de flujo de caja utilizado, se tiene un criterio con el cual evaluar diferentes escenarios las cuales sirven para orientar, de la mejor forma, decisiones futuras.

6.4 Conclusión general

En este proyecto, de forma integrada se abordó la optimización de una red de gasolineras con demanda estocástica mediante tres componentes acoplados: un modelo de demanda, un modelo de cadena de suministro y un modelo de flujo de caja. Se eligieron los modelos SARIMA porque pueden representar mejor los cambios repetitivos en el consumo de gasolina a lo largo del tiempo, se eligió un enfoque de optimización de cadena de suministro inspirado en modelos LRP y en técnicas metaheurísticas (AG + ACO), y un modelo estocástico multi-etapa apoyado en redes neuronales para la toma de decisiones en procesos con variables estocásticas.

Para la demanda estocástica, se inició considerando procesos modelados por Ornstein-Uhlenbeck y se concluyó que, debido a los patrones estacionales observados en los datos reales de consumo de combustibles, un modelo SARIMA es más adecuado para generar pronósticos de demanda por estación y periodo, que luego alimentan a los módulos de inventarios y flujo de caja. En la optimización temporal de la cadena de suministro, se formuló un problema de programación lineal que decide, para cada ciclo de reabastecimiento, cuánto entregar a cada una de las cinco estaciones y en qué momento, maximizando el tiempo entre recargas sujeto a restricciones de inventario mínimo, capacidad máxima de tanque y límite de 40 000 L del vehículo de transporte. Esta formulación se acopla a la demanda pronosticada y se activa cuando alguna estación alcanza su inventario mínimo, actualizando inventarios semana

a semana.

Finalmente, en la maximización del flujo de caja se implementó el modelo estocástico multi-etapa: para cada escenario de precios y trayectoria de demanda se modelan por periodo las compras, ventas e inventarios de cada estación, y se calcula el flujo de caja como ingresos menos egresos y costos de almacenamiento. A partir de los resultados gráficos, se observa que los inventarios promedio se mantienen por debajo de 10 000 L con picos asociados a recargas; que las ganancias totales son similares entre escenarios pero mayores cuando el precio de venta es más alto (escenario “caro”), aún con menor volumen vendido; y que la estación 4 presenta ventas y utilización de capacidad considerablemente menores al resto, lo que la vuelve poco atractiva para expandir capacidad en las condiciones actuales.

Como conclusión, se acoplaron las tres áreas mencionadas anteriormente para poder reflejar el comportamiento estocástico que se tiene en la demanda de la gasolina: la demanda alimenta las decisiones de inventario y éstas, a su vez, determinan los flujos de caja. Los resultados de cada submodelo con datos reales muestra que si se puede implementar una estrategia integrada para una red de gasolineras en donde se prioriza la disponibilidad del producto y la maximización del flujo de efectivo.

6.5 Conclusiones individuales

Diego Salinas: Este reto me permitió encontrar una aplicabilidad técnica a la solución de problemas por un lado estocástico. Además, nos desarrollamos de tal forma que aprendimos más sobre como combinar ecuaciones diferenciales en modelos estocásticos para poder así agregar una parte de ruido y conocer un comportamiento más realista del mismo. Logré aprender mucho de los procesos estocásticos y de los diferentes métodos para solucionar problemas. Sé que este reto me ayudó y permitirá mejorar competitivamente para aspirar a proyectos de alto valor en la sociedad.

Juan Eduardo Gonzalez: Particularmente, considero que fue una situación muy interesante en la cual nosotros como alumnos debimos adaptarnos a una problemática en la cual se tenían datos limitados y en base a ello realizar adecuaciones pertinentes. Me resulto bastante interesante el proceso de investigación y realización del modelo para llegar a resultados coherentes que nos permitiesen analizar la situación y dar soluciones, aunque sean parciales, recomendaciones y en base a ello adecuaciones que permitan mejoras significativas para nuestro socio formador.

Mariana Villarreal R. Vela: Este proyecto me permitió aprender más sobre diferentes modelos de op-

timización y cómo se pueden aplicar a problemas reales. Este tipo de retos me han ayudado a comprender que muchos problemas fuera de la escuela no tienen una única solución y que el enfoque es encontrar lo que se adapte mejor a los objetivos de quienes se beneficiarán de dicha solución.

En este proyecto me tocó trabajar principalmente en el problema con las cadenas de suministro. Entender que este es un problema de más de una variable y poder analizar todos los factores que lo involucran fue un reto bastante grande. Poder plantear una ecuación que modelara las variables necesarias para llegar a una optimización adecuada, nos llevó varios intentos. En cada uno de estos intentos aprendimos algo nuevo y pudimos adentrarnos en varios métodos de optimización hasta que hallamos el mejor para nuestro plantamiento del problema.

Tomamos varios caminos con diferentes enfoques, nuestras mejores opciones eran una ecuación diferencial y un modelo lineal. Optamos por el modelo lineal porque se ajustaba mejor a las restricciones planteadas y nuestro objetivo para optimizar el tiempo de reabastecimiento. Nuestra solución al problema fue minimizar la cantidad de residuo de combustible para que indirectamente se maximizara este tiempo de suministro.

Fue un reto interesante en el que aprendí muchas cosas y considero que hicimos un buen trabajo unificando las diferentes partes del trabajo.

Ximena Padrón Contreras : Dividir el reto en tres subgrupos me ayudó a entender mejor todo lo que se tuvo que considerar para poder modelar el problema de manera estructurada. Con la parte de flujo de caja, aprendí sobre algoritmos de optimización, como redes neuronales, algoritmos genéticos, y el algoritmo multi-etapa, lo cual me interesó mucho ya que ahora entiendo como estos se usan para describir procesos de la vida real, como la evolución de la demanda de la gasolina. Al final, el equipo de flujo de caja decidió implementar el método de multi etapa, la cual mejora la precisión de las predicciones y también es un algoritmo que se va adaptando ante variaciones inesperadas, maximizando la disponibilidad del producto. Con el algoritmo multi-etapa pudimos capturar la relación entre compras, ventas, y almacenamiento. Como conclusión, pude obtener un panorama computacional de la demanda estocástica de la gasolina que me permitió aprender cómo abordar este tipo de procesos.

Bernardo Lugo Albor: El proyecto me resultó muy interesante. Creo que el que no hubiera muchos datos disponibles por parte del socioformador nos impulsó a llevar a cabo un modelo de optimización más generalizado, que pudiera aplicarse a diferentes bases de datos a través de un mínimo de adaptación. También creo que el tener un equipo de trabajo grande y seccionado en diferentes áreas del proyecto fue muy útil, y

ayudó a que lográramos un producto final satisfactorio y que de verdad pudiera ayudar a optimizar una red gasolinera.

Jorge Luis Muñoz Reyes: Este trabajo me retó a buscar estrategias para superar los obstáculos que se presentan en un proyecto real de ciencia de datos como parte de una consultoría. Por ejemplo, uno de los mayores inconvenientes fue la falta de datos, pero con mi equipo logramos darle la vuelta utilizando bases de datos afines para recrear lo faltante y producir un modelo de calidad útil para el socioformador.

Hector Campbell: Encuentro satisfactorio el resultado y las conclusiones del proyecto. La implementación de procesos estocásticos para la proyección y optimización ayuda a contemplar sucesos inesperados y hacer una estrategia para la toma de decisiones de estos eventos. Es importante tomar en cuenta todas las variables y propiedades del proceso que queremos modelar, ya que estas nos encaminan a buscar la mejor forma de hacerlo. Nuestro modelo predictivo fue elaborado en gran medida por propiedades cualitativas que pudimos observar y dándole fundamento teórico y matemático. Estoy satisfecho con el resultado y su correcta implementación crea un proyecto de valor que es útil bajo diversos casos.

Alfredo Almaguer: Llevar a cabo este proyecto me permitió abordar un problema real a partir de utilizar los conocimientos que adquirí en el curso. Sin lugar a duda, la demanda y venta de gasolina es un proceso altamente estocástico que depende de una gran variedad de condiciones y parámetros. En este sentido, la estrategia que decidimos adoptar, fue basarnos en datos históricos con el objetivo de poder predecir los eventos a futuro. Trabajar en equipos grandes fue algo que nos permitió abordar el problema de manera más integral, de modo que cada sub-equipo logró dar solución a una fracción del problema global. Considero que trabajar en equipos grandes fue bastante retador y enriquecedor, ya que por un lado fue todo un desafío coordinarnos para solucionar el reto; pero por otro lado, escuchar distintos puntos de vista nos permitió tener un panorama más completo.

7 Referencias bibliográficas y trabajos citados

References

- [1] México Secretaría de Energía. Estadísticas de hidrocarburos. <https://estadisticashidrocarburos.energia.gob.mx/>, 2025. Sitio web oficial — consultado el 7 diciembre 2025.
- [2] Comisión Reguladora de Energía (CRE). Precios vigentes de gasolinas y diésel.

<https://www.gob.mx/cre/articulos/precios-vigentes-de-gasolinas-y-diesel>, 2025. Última consulta: 07 diciembre 2025.

- [3] M. E. Pate-Cornell, G. Tagaras, and K. M. Eisenhardt. Dynamic optimization of cash flow management decisions: a stochastic model. *IEEE Transactions on Engineering Management*, 37(3):203–212, 1990.
- [4] Claudio Turchetti. *Stochastic models of neural networks*, volume 102. IOS Press, 2004.
- [5] R Cuervo. Stochastic logistic models applied to the supplychain: A literature review. *UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS. Revista Ingeniería*, 26, 2021.
- [6] Paulina A. Avila-Torres, Nancy M. Arratia-Martinez, and Efraín Ruiz-y Ruiz. The inventory routing problem with priorities and fixed heterogeneous fleet. *Applied Sciences*, 10(10):3502, 2020.
- [7] J. García J. Coronado. Modelo estocástico para planificar cadenas de suministro con productos de ciclos de vida cortos. *Working Papers on Operations Management*, 1(1):1–5, 2010.
- [8] Juan Mascareñas and Marcelo Leporati. Opciones reales y flujo de caja descontado: ¿cuándo utilizarlos? Technical report, Observatorio de Divulgación Financiera — IEF, 2010. Nota Técnica No. 6.
- [9] M. Soto. Modelo estocástico y algoritmos en línea para el problema de gestión de saldo en caja en el banco central. *Cuestiones Económicas*, 23(1):86–86, 2007.
- [10] R. Tangsucheeva and V. Prabhu. Stochastic financial analytics for cash flow forecasting. *International Journal of Production Economics*, 158:65–76, 2014.
- [11] M. B. da Costa Moraes, M. S. Nagano, and V. A. Sobreiro. Stochastic cash flow management models: A literature review since the 1980s. In *Decision Models in Engineering and Management*, pages 11–28. 2015.

8 Anexos

Listing 1: Modelo de Optimización Estocástica para Red de Estaciones de Gasolina

```
"""
Modelo_de_Optimizaci_n_Estoc_stica_para_Red_de_Estaciones_de_Gasolina
-----
Usa:
- PreciosGasolinaRegular.xlsx
- PreciosGasolinaPremium.xlsx
- DieselPrecios.xlsx
- DatosSocioFormador.xlsx (capacidades, inventarios y consumos por estaci_n)

Hace:
- Construye par metros a partir de DatosSocioFormador.xlsx
- Resuelve modelo estoc stico multietapa
- Imprime resumen
- Genera gr_ficas de an_lisis
"""

# =====
#  INSTALACION EN COLAB
# =====
import sys
import subprocess

def install_dependencies():
    """Instala Pyomo y GLPK en Google Colab (si hace falta)."""
    try:
        import pyomo # noqa
        print("_Pyomo ya est _instalado")
    except ImportError:
        print("_Instalando Pyomo... ")
```

```

        subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "pyomo"])
        print("_Pyomo_instalado")

    print("_Instalando_solver_GLPK_(si_no_est...)")
    try:
        subprocess.check_call(["apt-get", "update", "-qq"])
        subprocess.check_call(["apt-get", "install", "-y", "-qq", "glpk-utils"])
        print("_GLPK_instalado_correctamente")
    except Exception as e:
        print("__Error_instalando_GLPK:", e)

def get_available_solver():
    """Forzamos_GLPK;_si_no_est_,_damos_un_mensaje_claro."""
    from pyomo.opt import SolverFactory
    solver_name = 'glpk'
    opt = SolverFactory(solver_name)
    if opt.available(False):
        print(f"_Usando_solver:{solver_name.upper()}")
        return solver_name
    else:
        raise RuntimeError(
            "_GLPK_no_est_disponible.\n"
            "En_Colab,_ejecuta_ANTES_de_este_codigo:\n\n"
            "__!apt-get_update_-qq\n"
            "__!apt-get_install_-y_-qq_glpk-utils\n"
            "__!pip_install_-q_pyomo\n"
        )

# Ejecutar instalaci3n auto si estamos en Colab
if 'google.colab' in sys.modules:
    print("_Configurando_entorno_de_Google_Colab...")
    install_dependencies()
    print("_Configuraci3n_completa\n")

# =====
# IMPORTS PRINCIPALES
# =====
from pyomo.environ import (
    ConcreteModel, Var, NonNegativeReals, Set, Param,
    RangeSet, Objective, Constraint, ConstraintList,
    maximize, value
)
from pyomo.opt import SolverFactory, TerminationCondition

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.gridspec import GridSpec

sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (14, 10)
plt.rcParams['font.size'] = 10

# =====
# MODELO PYOMO
# =====

```

```

def build_multi_station_model(
    T, stations, escenarios, prob, demand, price_buy, price_sell,
    fixed_cost, tank_capacity, holding_cost, initial_inventory,
    na_groups=None
):
    m = ConcreteModel("Red_Gasolineras")

    # Conjuntos
    m.T = RangeSet(1, T, doc="Periodos_de_tiempo")
    m.J = Set(initialize=stations, doc="Estaciones")
    m.S = Set(initialize=escenarios, doc="Escenarios")

    # Par metros
    m.prob = Param(m.S, initialize=prob, doc="Probabilidad_de_escenario")
    m.demand = Param(m.T, m.J, m.S, initialize=demand, doc="Demanda")
    m.p_buy = Param(m.T, m.J, m.S, initialize=price_buy, doc="Precio_compra")
    m.p_sell = Param(m.T, m.J, m.S, initialize=price_sell, doc="Precio_venta")
    m.fixed_cost = Param(m.T, m.J, initialize=fixed_cost, doc="Costo_fijo")
    m.tank_capacity = Param(m.J, initialize=tank_capacity, doc="Capacidad_tanque")
    m.holding_cost = Param(m.J, initialize=holding_cost, doc="Costo_almacenamiento")
    m.I0 = Param(m.J, initialize=initial_inventory, doc="Inventario_inicial")

    # Variables
    m.buy = Var(m.T, m.J, m.S, domain=NonNegativeReals, doc="Cantidad_a_comprar")
    m.sell = Var(m.T, m.J, m.S, domain=NonNegativeReals, doc="Cantidad_a_vender")
    m.inv = Var(m.T, m.J, m.S, domain=NonNegativeReals, doc="Inventario_final")

    # Balance de inventario
    def inventory_balance_rule(m, t, j, s):
        if t == 1:
            return m.inv[t, j, s] == m.I0[j] + m.buy[t, j, s] - m.sell[t, j, s]
        else:
            return m.inv[t, j, s] == m.inv[t-1, j, s] + m.buy[t, j, s] - m.sell[t, j, s]
    m.inventory_balance = Constraint(m.T, m.J, m.S, rule=inventory_balance_rule)

    # Capacidad de tanque
    def capacity_rule(m, t, j, s):
        return m.inv[t, j, s] <= m.tank_capacity[j]
    m.capacity = Constraint(m.T, m.J, m.S, rule=capacity_rule)

    # Ventas demanda
    def demand_rule(m, t, j, s):
        return m.sell[t, j, s] <= m.demand[t, j, s]
    m.sales_demand = Constraint(m.T, m.J, m.S, rule=demand_rule)

    # Ventas disponibilidad
    def availability_rule(m, t, j, s):
        if t == 1:
            return m.sell[t, j, s] <= m.I0[j] + m.buy[t, j, s]
        else:
            return m.sell[t, j, s] <= m.inv[t-1, j, s] + m.buy[t, j, s]
    m.sales_avail = Constraint(m.T, m.J, m.S, rule=availability_rule)

    # No-anticipatividad
    m.NA = ConstraintList()
    if na_groups is not None:

```

```

    for t, groups in na_groups.items():
        for group in groups:
            if len(group) <= 1:
                continue
            first = group[0]
            for s in group[1:]:
                for j in m.J:
                    m.NA.add(m.buy[t, j, s] == m.buy[t, j, first])

# Objetivo: maximizar ganancia esperada
def objective_rule(m):
    return sum(
        m.prob[s] * sum(
            m.p_sell[t, j, s] * m.sell[t, j, s]
            - m.p_buy[t, j, s] * m.buy[t, j, s]
            - m.holding_cost[j] * m.inv[t, j, s]
            - m.fixed_cost[t, j]
            for t in m.T
            for j in m.J
        )
        for s in m.S
    )
m.profit = Objective(rule=objective_rule, sense=maximize)

return m

```

```

# =====
#  UTILIDADES DE RESULTADOS
# =====

```

```

def extract_results(model):
    results = {
        'optimal_profit': value(model.profit),
        'decisions': [],
        'summary_by_station': {},
        'summary_by_scenario': {}
    }

    for s in model.S:
        for j in model.J:
            for t in model.T:
                results['decisions'].append({
                    'scenario': s,
                    'station': j,
                    'period': t,
                    'buy': value(model.buy[t, j, s]),
                    'sell': value(model.sell[t, j, s]),
                    'inventory': value(model.inv[t, j, s])
                })

    for j in model.J:
        total_buy = sum(value(model.prob[s]) * sum(value(model.buy[t, j, s])
            for t in model.T) for s in model.S)
        total_sell = sum(value(model.prob[s]) * sum(value(model.sell[t, j, s])
            for t in model.T) for s in model.S)
        avg_inv = sum(value(model.prob[s]) * sum(value(model.inv[t, j, s])

```

```

        for t in model.T) for s in model.S) / len(model.T)

    results['summary_by_station'][j] = {
        'total_buy': total_buy,
        'total_sell': total_sell,
        'avg_inventory': avg_inv
    }

for s in model.S:
    total_buy = sum(value(model.buy[t, j, s]) for t in model.T for j in model.J)
    total_sell = sum(value(model.sell[t, j, s]) for t in model.T for j in model.J)

    results['summary_by_scenario'][s] = {
        'total_buy': total_buy,
        'total_sell': total_sell,
        'probability': value(model.prob[s])
    }

return results

def print_results(results):
    print("\n" + "-"*70)
    print(f"{'RESULTADOS_DE_OPTIMIZACION':^70}")
    print("-"*70)
    print(f"\n_Ganancia_Esperada_ ptima : {results['optimal_profit']: .2f}")

    print("\n" + "-"*70)
    print(f"{'RESUMEN_POR_ESTACION':^70}")
    print("-"*70)
    print(f"{'Estacion':<12}{'Compra_Total':>15}{'Venta_Total':>15}{'Inv._Promedio':>15}")
    print("-"*70)

    for station, data in results['summary_by_station'].items():
        print(f"{'station':<12}{'data['total_buy']':>15.1f}{'data['total_sell']':>15.1f}{'data['avg_inventory']':>15.1f}")

    print("\n" + "-"*70)
    print(f"{'RESUMEN_POR_ESCENARIO':^70}")
    print("-"*70)
    print(f"{'Escenario':<15}{'Probabilidad':>15}{'Compra_Total':>15}{'Venta_Total':>15}")
    print("-"*70)

    for scenario, data in results['summary_by_scenario'].items():
        print(f"{'scenario':<15}{'data['probability']':>15.2%}{'data['total_buy']':>15.1f}{'data['total_sell']':>15.1f}")
    print("-"*70 + "\n")

# =====
# FUNCIONES DE GRFICAS
# =====

def plot_comprehensive_analysis(model, results):
    fig = plt.figure(figsize=(16, 12))
    gs = GridSpec(3, 3, figure=fig, hspace=0.3, wspace=0.3)

```

```

# 1. Ganancia por estaci n
ax1 = fig.add_subplot(gs[0, :2])
station_data = results['summary_by_station']
stations_plot = list(station_data.keys())
revenues = []

for station in stations_plot:
    rev = sum(value(model.prob[s]) * sum(
        value(model.p_sell[t, station, s]) * value(model.sell[t, station, s])
        - value(model.p_buy[t, station, s]) * value(model.buy[t, station, s])
        - value(model.holding_cost[station]) * value(model.inv[t, station, s])
        - value(model.fixed_cost[t, station])
        for t in model.T
    ) for s in model.S)
    revenues.append(rev)

colors = plt.cm.viridis(np.linspace(0.2, 0.8, len(stations_plot)))
bars = ax1.bar(stations_plot, revenues, color=colors, edgecolor='black', linewidth=1.2)
ax1.set_title('_Ganancia_Esperada_por_Estaci n', fontsize=14, fontweight='bold', pad=1)
ax1.set_xlabel('Estaci n', fontsize=11)
ax1.set_ylabel('Ganancia', fontsize=11)
ax1.axhline(y=0, color='red', linestyle='—', linewidth=0.8, alpha=0.5)
ax1.grid(True, alpha=0.3)

for bar in bars:
    height = bar.get_height()
    ax1.text(bar.get_x() + bar.get_width()/2., height,
             f'{height:,.0f}',
             ha='center', va='bottom' if height > 0 else 'top', fontsize=9)

# 2. Compras vs ventas por estaci n
ax2 = fig.add_subplot(gs[0, 2])
buy_volumes = [station_data[s]['total_buy'] for s in stations_plot]
sell_volumes = [station_data[s]['total_sell'] for s in stations_plot]

x = np.arange(len(stations_plot))
width = 0.35
ax2.barh(x - width/2, buy_volumes, width, label='Compras')
ax2.barh(x + width/2, sell_volumes, width, label='Ventas')
ax2.set_yticks(x)
ax2.set_yticklabels(stations_plot)
ax2.set_xlabel('Volumen', fontsize=10)
ax2.set_title('_Compras_vs_Ventas', fontsize=12, fontweight='bold')
ax2.legend(loc='lower_right')
ax2.grid(True, alpha=0.3, axis='x')

# 3. Inventario total por escenario
ax3 = fig.add_subplot(gs[1, :])
df_dec = pd.DataFrame(results['decisions'])

for s in model.S:
    df_scenario = df_dec[df_dec['scenario'] == s]
    inv_by_period = df_scenario.groupby('period')['inventory'].sum()
    prob = value(model.prob[s])
    ax3.plot(inv_by_period.index, inv_by_period.values,
            marker='o', linewidth=2.5, markersize=6,
            label=f'Escenario_{s}_ (p={prob:.1%})')

```

```

ax3.set_title('_', 'Inventario_total_por_escenario', fontsize=14, fontweight='bold', pad=15)
ax3.set_xlabel('Periodo', fontsize=11)
ax3.set_ylabel('Inventario_total', fontsize=11)
ax3.legend(loc='best', fontsize=10)
ax3.grid(True, alpha=0.3)

# 4. Heatmap de compras para un escenario
ax4 = fig.add_subplot(gs[2, 0])
scenario_choice = list(model.S)[0]
df_buy = df_dec[df_dec['scenario'] == scenario_choice].pivot(
    index='station', columns='period', values='buy'
)
sns.heatmap(df_buy, annot=False, fmt='.0f', cmap='YlOrRd',
            ax=ax4, cbar_kws={'label': 'Unidades'})
ax4.set_title(f'Compras_-_Escenario_{scenario_choice}', fontsize=12, fontweight='bold')
ax4.set_xlabel('Periodo', fontsize=10)
ax4.set_ylabel('Estaci n', fontsize=10)

# 5. Heatmap de ventas
ax5 = fig.add_subplot(gs[2, 1])
df_sell = df_dec[df_dec['scenario'] == scenario_choice].pivot(
    index='station', columns='period', values='sell'
)
sns.heatmap(df_sell, annot=False, fmt='.0f', cmap='Greens',
            ax=ax5, cbar_kws={'label': 'Unidades'})
ax5.set_title(f'Ventas_-_Escenario_{scenario_choice}', fontsize=12, fontweight='bold')
ax5.set_xlabel('Periodo', fontsize=10)
ax5.set_ylabel('Estaci n', fontsize=10)

# 6. Utilizaci n de capacidad
ax6 = fig.add_subplot(gs[2, 2])
utilization = []
for station in stations_plot:
    max_inv = max(value(model.inv[t, station, s])
                  for t in model.T for s in model.S)
    capacity = value(model.tank_capacity[station])
    utilization.append((max_inv / capacity) * 100)

colors_util = ['#2ECC71' if u < 80 else '#F39C12' if u < 95 else '#E74C3C'
               for u in utilization]
ax6.barh(stations_plot, utilization, color=colors_util, edgecolor='black')
ax6.set_xlabel('Utilizaci n (%)', fontsize=10)
ax6.set_title('_', 'Utilizaci n_de_capacidad', fontsize=12, fontweight='bold')
ax6.axvline(x=80, color='orange', linestyle='—', linewidth=1, alpha=0.7)
ax6.axvline(x=95, color='red', linestyle='—', linewidth=1, alpha=0.7)
ax6.grid(True, alpha=0.3, axis='x')

for i, v in enumerate(utilization):
    ax6.text(v + 1, i, f'{v:.1f}%', va='center', fontsize=9)

plt.suptitle('_', 'An lisis_integral_de_la_red_de_estaciones', fontsize=16,
            fontweight='bold', y=0.995)
plt.tight_layout()
return fig

```

```

def plot_scenario_comparison(model, results):
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle('_Comparaci n_entre_escenarios', fontsize=16, fontweight='bold')

    df_dec = pd.DataFrame(results['decisions'])
    scenarios = list(model.S)

    # 1. Compras totales por periodo
    ax1 = axes[0, 0]
    for s in scenarios:
        df_s = df_dec[df_dec['scenario'] == s]
        buy_by_period = df_s.groupby('period')['buy'].sum()
        prob = value(model.prob[s])
        ax1.plot(buy_by_period.index, buy_by_period.values,
                 marker='s', linewidth=2, markersize=6,
                 label=f'{s}_ (p={prob:.1%})')
    ax1.set_title('Compras_totales_por_periodo', fontweight='bold')
    ax1.set_xlabel('Periodo')
    ax1.set_ylabel('Unidades_compradas')
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # 2. Ventas totales por periodo
    ax2 = axes[0, 1]
    for s in scenarios:
        df_s = df_dec[df_dec['scenario'] == s]
        sell_by_period = df_s.groupby('period')['sell'].sum()
        prob = value(model.prob[s])
        ax2.plot(sell_by_period.index, sell_by_period.values,
                 marker='o', linewidth=2, markersize=6,
                 label=f'{s}_ (p={prob:.1%})')
    ax2.set_title('Ventas_totales_por_periodo', fontweight='bold')
    ax2.set_xlabel('Periodo')
    ax2.set_ylabel('Unidades_vendidas')
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    # 3. Distribuci n de inventarios
    ax3 = axes[1, 0]
    inv_data = [df_dec[df_dec['scenario'] == s]['inventory'].values for s in scenarios]
    bp = ax3.boxplot(inv_data, labels=scenarios, patch_artist=True)
    for patch, color in zip(bp['boxes'], ['#3498DB', '#E74C3C', '#2ECC71'][:len(scenarios)]):
        patch.set_facecolor(color)
        patch.set_alpha(0.6)
    ax3.set_title('Distribuci n_de_inventarios', fontweight='bold')
    ax3.set_ylabel('Nivel_de_inventario')
    ax3.grid(True, alpha=0.3, axis='y')

    # 4. Ganancia por escenario
    ax4 = axes[1, 1]
    profits = []
    for s in scenarios:
        profit_s = sum(
            value(model.p_sell[t, j, s]) * value(model.sell[t, j, s])
            - value(model.p_buy[t, j, s]) * value(model.buy[t, j, s])
            - value(model.holding_cost[j]) * value(model.inv[t, j, s])
            - value(model.fixed_cost[t, j])
        )
        profits.append(profit_s)
    ax4.set_title('Ganancia_por_escenario', fontweight='bold')
    ax4.set_xlabel('Periodo')
    ax4.set_ylabel('Ganancia')
    ax4.grid(True, alpha=0.3)

```

```

        for t in model.T for j in model.J
    )
    profits.append(profit_s)

colors_profit = ['#2ECC71' if p > 0 else '#E74C3C' for p in profits]
bars = ax4.bar(scenarios, profits, color=colors_profit,
               edgecolor='black', linewidth=1.5)
ax4.set_title('Ganancia_por_escenario', fontweight='bold')
ax4.set_ylabel('Ganancia')
ax4.axhline(y=0, color='black', linestyle='-', linewidth=0.8)
ax4.grid(True, alpha=0.3, axis='y')

for bar, profit in zip(bars, profits):
    height = bar.get_height()
    ax4.text(bar.get_x() + bar.get_width()/2., height,
             f'{profit:,.0f}',
             ha='center', va='bottom' if height > 0 else 'top',
             fontsize=11, fontweight='bold')

plt.tight_layout()
return fig

# =====
#  MAIN:  USO DE TUS EXCEL
# =====
if __name__ == "__main__":
    print("\n" + "Gasolinera_"*20)
    print("_OPTIMIZACION_CON_DATOS_REALES_DE_PRECIOS_Y_SOCIO_FORMADOR")
    print("Gasolinera_"*20 + "\n")

    # ===== 1. RUTA DE ARCHIVOS =====
    # Ajusta la ruta si tus archivos est n en otra carpeta
    ruta = "" # por ejemplo: '/content/drive/MyDrive/CarpetaGasolinera/'

    # ===== 2. LEER PRECIOS =====
    df_reg = pd.read_excel(ruta + "PreciosGasolinaRegular.xlsx")
    df_prem = pd.read_excel(ruta + "PreciosGasolinaPremium.xlsx")
    df_dies = pd.read_excel(ruta + "DieselPrecios.xlsx")

    df_precios = df_reg.merge(df_prem, on="fecha").merge(df_dies, on="fecha")
    df_precios = df_precios.sort_values("fecha").reset_index(drop=True)

    print("Primeras_filas_de_datos_de_precios:")
    print(df_precios.head())

    T = len(df_precios) # cada fila es un periodo

    # ===== 3. LEER DATOS SOCIO FORMADOR =====
    df_sf = pd.read_excel(ruta + "DatosSocioFormador.xlsx")

    # Buscar filas con "Estacion X" (1..5)
    station_rows = []
    for i, val in enumerate(df_sf['Unnamed:0']):
        if isinstance(val, str) and val.startswith("Estacion_") and "promedios" not in val:
            station_rows.append(i)

```

```

stations = []
tank_capacity = {}
initial_inventory = {}
base_demanda_estacion = {}

for idx in station_rows:
    name = df_sf.loc[idx, 'Unnamed: 0'] # 'Estacion 1'
    try:
        num = int(name.split()[1])
    except Exception:
        num = len(stations) + 1
    j = f"E{num}"
    stations.append(j)

    header_row = idx + 1 # nombres de productos
    cap_row = idx + 2 # 'Capacidad'
    actual_row = idx + 3 # 'Actual'
    consumo_row = idx + 5 # 'Consumo' (idx+4 es 'Dias')

    # Productos en columnas 1..3
    prod_cols = df_sf.columns[1:4]
    valid_cols = []
    for c in prod_cols:
        prod_name = df_sf.loc[header_row, c]
        if isinstance(prod_name, str) and prod_name.strip() in ("Regular", "Premium", "
            valid_cols.append(c)

    cap_vals = [df_sf.loc[cap_row, c] for c in valid_cols]
    act_vals = [df_sf.loc[actual_row, c] for c in valid_cols]
    cons_vals = [df_sf.loc[consumo_row, c] for c in valid_cols]

    tank_capacity[j] = float(np.nansum(cap_vals))
    initial_inventory[j] = float(np.nansum(act_vals))
    base_demanda_estacion[j] = float(np.nansum(cons_vals))

stations = sorted(stations)
print("\nEstaciones detectadas y par metros desde DatosSocioFormador:")
for j in stations:
    print(f"{j}: Capacidad={tank_capacity[j]:.1f}, Inventario inicial={initial_inventor
        f"Demanda base {base_demanda_estacion[j]:.1f}")

# ===== 4. ESCENARIOS Y DEMANDA =====
scenarios = ['base', 'caro', 'barato']
prob = {'base': 0.5, 'caro': 0.25, 'barato': 0.25}

demand = {}
price_buy = {}
price_sell = {}

precios_reg = df_precios['Gasolina Regular'].values
precio_medio = precios_reg.mean()

for t in range(1, T + 1):
    row = df_precios.iloc[t - 1]
    precio_reg_t = row['Gasolina Regular']

    for j in stations:

```

```

base_dem = base_demanda_estacion[j]

for s in escenarios:
    if s == 'base':
        p_sell = precio_reg_t
        p_buy = p_sell * 0.95
        factor = max(0.3, 1 - 0.10 * (p_sell - precio_medio) / precio_medio)
    elif s == 'caro':
        p_sell = precio_reg_t * 1.05
        p_buy = p_sell * 0.95
        factor = max(0.3, 1 - 0.15 * (p_sell - precio_medio) / precio_medio)
    else: # 'barato'
        p_sell = precio_reg_t * 0.95
        p_buy = p_sell * 0.95
        factor = max(0.3, 1 - 0.05 * (p_sell - precio_medio) / precio_medio)

dem = base_dem * factor

price_sell[(t, j, s)] = float(p_sell)
price_buy[(t, j, s)] = float(p_buy)
demand[(t, j, s)] = float(dem)

# ===== 5. OTROS PARAMETROS =====
fixed_cost = {}
for t in range(1, T + 1):
    for j in stations:
        # Aqu podr as sustituir por un costo fijo real mensual/semanal
        fixed_cost[(t, j)] = 1000.0 + 50.0 * int(j[1:])

holding_cost = {j: 0.05 + 0.005 * int(j[1:]) for j in stations}

# No-anticipatividad: mismas compras en t=1 en todos los escenarios
na_groups = {1: [scenarios]}

# ===== 6. CONSTRUIR Y RESOLVER MODELO =====
print("\n_Construyendo_modelo_de_optimizaci n...")
model = build_multi_station_model(
    T, stations, escenarios, prob, demand, price_buy, price_sell,
    fixed_cost, tank_capacity, holding_cost, initial_inventory,
    na_groups=na_groups
)

print("_Detectando_solver_disponible...")
solver_name = get_available_solver()
opt = SolverFactory(solver_name)

print(f"_Resolviendo_modelo_con_{solver_name.upper()}...\n")
result = opt.solve(model, tee=False)

if result.solver.termination_condition == TerminationCondition.optimal:
    print("_Soluci n ptima _encontrada!\n")
    results = extract_results(model)
    print_results(results)

    print("_Generando_gr ficas...\n")
    fig1 = plot_comprehensive_analysis(model, results)
    plt.show()

```

```

fig2 = plot_scenario_comparison(model, results)
plt.show()

print("\n Análisis completo finalizado.")
else:
    print(f"Solver terminado con estado: {result.solver.termination_condition}")

```

Listing 2: Simulación y optimización del inventario en estaciones de servicio

```

import numpy as np
import pandas as pd
import cvxpy as cp
import matplotlib.pyplot as plt

!pip install ecos

# (1) CARGAR PRONSTICOS SEMANALES DESDE EL ARCHIVO SARIMA

ruta_excel = r"resultados_estaciones_(1).xlsx"

df_proyecciones = pd.read_excel(
    ruta_excel,
    sheet_name="Proyecciones",
    index_col=0
)

n = 5 # n mero de estaciones
T_total = len(df_proyecciones)

# Matriz de medias:  $\mu_i(t)$ 
mu = df_proyecciones.values.T # shape: (n, T_total)

# (2) ESTIMAR  $\sigma_i(t)$  A PARTIR DE LA VARIABILIDAD DEL PRONSTICO

sigma = np.zeros_like(mu)

for i in range(n):
    serie = df_proyecciones.iloc[:, i]
    # desviación estándar móvil semanal
    roll_std = serie.diff().rolling(4).std().fillna(serie.std())
    sigma[i] = np.maximum(roll_std.values, 1.0)

# (3) CONFIGURACION DEL MODELO DE INVENTARIO (TODO SEMANAL)

I_min = np.array([10000, 12000, 9000, 11000, 10000], dtype=float)
I_max = np.array([68991.05, 55981.45, 49611.06, 24046.678, 51829.93], dtype=float)
I = np.array([20000, 20000, 20000, 20000, 20000], dtype=float) # Inventario inicial

C = 40000 # capacidad total del camión

I_hist = [] # para graficar inventarios
refill_weeks = [] # para marcar las semanas de reabastecimiento

```

```
# (4) OPTIMIZACIN SEMANAL (CVXPY – PROGRAMACIN LINEAL)
```

```
def optimize_refill(I_current, mu_current):
    Q = cp.Variable(n, nonneg=True)
    T = cp.Variable(nonneg=True)

    constraints = []

    constraints.append(I_current + Q - mu_current * T >= I_min)
    constraints.append(I_current + Q <= I_max)
    constraints.append(cp.sum(Q) <= C)

    objective = cp.Maximize(T)

    prob = cp.Problem(objective, constraints)
    prob.solve(solver=cp.ECOS)

    return Q.value, T.value
```

```
# (5) SIMULACIN SEMANAL ESTOCSTICA + OPTIMIZACIN
```

```
for t in range(T_total):

    I_hist.append(I.copy())

    # Demanda real semanal
    d_real = np.random.normal(mu[:, t], sigma[:, t])
    d_real = np.maximum(d_real, 0)

    # Consumir inventario
    I = I - d_real

    # Si cae al m nimo reabastecer
    if np.any(I <= I_min):
        print(f"\n_RÉABASTECIMIENTO_en_semana_{t}")
        print("Inventario_antes_del_refill:", I)

        Q_opt, T_opt = optimize_refill(I, mu[:, t])

        print("Q_ ptimos (litros):", np.round(Q_opt, 2))
        print("Tiempo_ ptimo :", T_opt)

        refill_weeks.append(t)

    # Reabastecer
    I = I + Q_opt
    print("Inventario_despu s:", I)

I_hist = np.array(I_hist)
```

```
# (6) GR FICA CONSOLIDADA
```

```
plt.figure(figsize=(15, 7))
```

```
for i in range(n):  
    plt.plot(I_hist[:, i], label=f"Estaci n_{i+1}", linewidth=2)
```

```
# Marcar reabastecimientos
```

```
for w in refill_weeks:  
    plt.axvline(x=w, color='gray', linestyle='—', alpha=0.5)
```

```
plt.title("Inventarios_semanales_(todas_las_estaciones)")  
plt.xlabel("Semana")  
plt.ylabel("Inventario_(litros)")  
plt.grid(True)  
plt.legend()  
plt.show()
```

```
# (7) GR FICAS INDIVIDUALES POR ESTACION
```

```
for i in range(n):  
    plt.figure(figsize=(12, 5))  
  
    plt.plot(I_hist[:, i], label=f"Inventario_estaci n_{i+1}",  
            linewidth=2, color='blue')
```

```
# L neas horizontales de cotas
```

```
plt.axhline(I_min[i], color='red', linestyle='—',  
            label='Inventario_m nimo')  
plt.axhline(I_max[i], color='green', linestyle='—',  
            label='Inventario_m ximo')
```

```
# L neas verticales donde hubo reabastecimiento
```

```
for w in refill_weeks:  
    plt.axvline(x=w, color='black', linestyle=':', alpha=0.6)
```

```
plt.title(f"Evoluci n_del_inventario_-_Estaci n_{i+1}")  
plt.xlabel("Semana")  
plt.ylabel("Inventario_(litros)")  
plt.grid(True)  
plt.legend()  
plt.tight_layout()  
plt.show()
```

```
1 # -*- coding: utf-8 -*-  
2 """reabastecimiento.ipynb  
3  
4 Automatically generated by Colab.  
5  
6 Original file is located at  
7 https://colab.research.google.com/drive/1e-w0008PBAhsq\_0hokdLmjZA26Wghq4-  
8 """  
9  
10 import numpy as np
```

```

11 import pandas as pd
12 import cvxpy as cp
13 import matplotlib.pyplot as plt
14
15 !pip install ecos
16
17 # (1) CARGAR PRONÓSTICOS SEMANALES DESDE EL ARCHIVO SARIMA
18
19 ruta_excel = r"resultados_estaciones (1).xlsx"
20
21 df_proyecciones = pd.read_excel(
22     ruta_excel,
23     sheet_name="Proyecciones",
24     index_col=0
25 )
26
27 n = 5 # número de estaciones
28 T_total = len(df_proyecciones)
29
30 # Matriz de medias:  $\mu_i(t)$ 
31 mu = df_proyecciones.values.T # shape: (n, T_total)
32
33
34
35 # (2) ESTIMAR  $\mu_i(t)$  A PARTIR DE LA VARIABILIDAD DEL PRONÓSTICO
36
37 sigma = np.zeros_like(mu)
38
39 for i in range(n):
40     serie = df_proyecciones.iloc[:, i]
41     # desviación estándar móvil semanal
42     roll_std = serie.diff().rolling(4).std().fillna(serie.std())
43     sigma[i] = np.maximum(roll_std.values, 1.0)
44
45
46
47 # (3) CONFIGURACIÓN DEL MODELO DE INVENTARIO (TODO SEMANAL)
48 # (CAPACIDADES EXCEL SOCIOFORMADOR)
49
50 I_min = np.array([10000, 12000, 9000, 11000, 10000], dtype=float)
51 I_max = np.array([68991.05, 55981.45, 49611.06, 24046.678, 51829.93], dtype=float)
52 I = np.array([20000, 20000, 20000, 20000, 20000], dtype=float) # Inventario inicial
53
54 C = 40000 # capacidad total del camión
55 I_hist = [] # para graficar
56
57
58
59 # (4) OPTIMIZACIÓN SEMANAL (CVXPY - PROGRAMACIÓN LINEAL)
60
61
62 def optimize_refill(I_current, mu_current):
63     """
64     Resuelve el LP usando medias semanales pronosticadas.
65     """
66     Q = cp.Variable(n, nonneg=True) # litros a entregar esta semana
67     T = cp.Variable(nonneg=True) # semanas antes del siguiente reabastecimiento
68

```

```

69     constraints = []
70
71     # No caer por debajo del mínimo dentro de T semanas
72     #  $I_i(0) + Q_i - \mu_i * T \geq I_{min_i}$ 
73     constraints.append(I_current + Q - mu_current * T >= I_min)
74
75     # Respeto del inventario máximo físico
76     constraints.append(I_current + Q <= I_max)
77
78     # Capacidad total del camión
79     constraints.append(cp.sum(Q) <= C)
80
81     objective = cp.Maximize(T)
82
83     problem = cp.Problem(objective, constraints)
84     problem.solve(solver=cp.ECOS)
85
86     return Q.value, T.value
87
88
89
90 # (5) SIMULACIÓN SEMANAL ESTOCÁSTICA + OPTIMIZACIÓN
91
92
93 for t in range(T_total):
94
95     I_hist.append(I.copy()) # guardar inventarios actuales
96
97     # Demanda REAL de la semana =  $N(\mu_i(t), \sigma_i(t))$ 
98     d_real = np.random.normal(mu[:, t], sigma[:, t])
99     d_real = np.maximum(d_real, 0)
100
101     # Consumir inventario
102     I = I - d_real
103
104     # Si alguna estación cae al inventario mínimo → reabastecer
105     if np.any(I <= I_min):
106         print(f"\n REABASTECIMIENTO en semana {t}")
107         print("Inventario antes del refill:", I)
108
109         Q_opt, T_opt = optimize_refill(I, mu[:, t])
110
111         print("Q óptimos (litros):", np.round(Q_opt, 2))
112         print("Tiempo óptimo (semanas hasta el siguiente refill):", T_opt)
113
114         # Aplicar reabastecimiento
115         I = I + Q_opt
116
117         print("Inventario después del refill:", I)
118
119
120 I_hist = np.array(I_hist)
121
122
123
124 # (6) GRÁFICAS REABASTECIMIENTO PERIÓDICO
125
126

```

```

127 plt.figure(figsize=(14, 7))
128 for i in range(n):
129     plt.plot(I_hist[:, i], label=f"Estación {i+1}", linewidth=2)
130
131 plt.title("Inventarios semanales simulados con demanda SARIMA + estocástica")
132 plt.xlabel("Semana")
133 plt.ylabel("Inventario (litros)")
134 plt.grid(True)
135 plt.legend()
136 plt.show()
137
138 import numpy as np
139 import pandas as pd
140 import cvxpy as cp
141 import matplotlib.pyplot as plt
142
143 !pip install ecos
144
145 # (1) CARGAR PRONÓSTICOS SEMANALES DESDE EL ARCHIVO SARIMA
146
147 ruta_excel = r"resultados_estaciones (1).xlsx"
148
149 df_proyecciones = pd.read_excel(
150     ruta_excel,
151     sheet_name="Proyecciones",
152     index_col=0
153 )
154
155 n = 5 # número de estaciones
156 T_total = len(df_proyecciones)
157
158 # Matriz de medias:  $\mu_i(t)$ 
159 mu = df_proyecciones.values.T # shape: (n, T_total)
160
161
162
163 # (2) ESTIMAR  $\sigma_i(t)$  A PARTIR DE LA VARIABILIDAD DEL PRONÓSTICO
164
165 sigma = np.zeros_like(mu)
166
167 for i in range(n):
168     serie = df_proyecciones.iloc[:, i]
169     # desviación estándar móvil semanal
170     roll_std = serie.diff().rolling(4).std().fillna(serie.std())
171     sigma[i] = np.maximum(roll_std.values, 1.0)
172
173
174
175 # (3) CONFIGURACIÓN DEL MODELO DE INVENTARIO (TODO SEMANAL)
176
177 I_min = np.array([10000, 12000, 9000, 11000, 10000], dtype=float)
178 I_max = np.array([68991.05, 55981.45, 49611.06, 24046.678, 51829.93], dtype=float)
179 I = np.array([20000, 20000, 20000, 20000, 20000], dtype=float) # Inventario inicial
180
181 C = 40000 # capacidad total del camión
182
183 I_hist = [] # para graficar inventarios
184 refill_weeks = [] # para marcar las semanas de reabastecimiento

```

```

185
186
187
188 # (4) OPTIMIZACIÓN SEMANAL (CVXPY - PROGRAMACIÓN LINEAL)
189
190 def optimize_refill(I_current, mu_current):
191     Q = cp.Variable(n, nonneg=True)
192     T = cp.Variable(nonneg=True)
193
194     constraints = []
195
196     constraints.append(I_current + Q - mu_current * T >= I_min)
197     constraints.append(I_current + Q <= I_max)
198     constraints.append(cp.sum(Q) <= C)
199
200     objective = cp.Maximize(T)
201
202     prob = cp.Problem(objective, constraints)
203     prob.solve(solver=cp.ECOS)
204
205     return Q.value, T.value
206
207
208
209 # (5) SIMULACIÓN SEMANAL ESTOCÁSTICA + OPTIMIZACIÓN
210
211 for t in range(T_total):
212
213     I_hist.append(I.copy())
214
215     # Demanda real semanal
216     d_real = np.random.normal(mu[:, t], sigma[:, t])
217     d_real = np.maximum(d_real, 0)
218
219     # Consumir inventario
220     I = I - d_real
221
222     # Si cae al mínimo → reabastecer
223     if np.any(I <= I_min):
224         print(f"\n REABASTECIMIENTO en semana {t}")
225         print("Inventario antes del refill:", I)
226
227         Q_opt, T_opt = optimize_refill(I, mu[:, t])
228
229         print("Q óptimos (litros):", np.round(Q_opt, 2))
230         print("Tiempo óptimo:", T_opt)
231
232         refill_weeks.append(t)
233
234         # Reabastecer
235         I = I + Q_opt
236         print("Inventario después:", I)
237
238
239 I_hist = np.array(I_hist)
240
241
242

```

```

243
244 # (6) GRÁFICA CONSOLIDADA
245
246
247 plt.figure(figsize=(15, 7))
248
249 for i in range(n):
250     plt.plot(I_hist[:, i], label=f"Estación {i+1}", linewidth=2)
251
252 # Marcar reabastecimientos
253 for w in refill_weeks:
254     plt.axvline(x=w, color='gray', linestyle='--', alpha=0.5)
255
256 plt.title("Inventarios semanales (todas las estaciones)")
257 plt.xlabel("Semana")
258 plt.ylabel("Inventario (litros)")
259 plt.grid(True)
260 plt.legend()
261 plt.show()
262
263
264
265
266 # (7) GRÁFICAS INDIVIDUALES POR ESTACIÓN
267
268
269 for i in range(n):
270     plt.figure(figsize=(12, 5))
271
272     plt.plot(I_hist[:, i], label=f"Inventario estación {i+1}", linewidth=2, color='blue')
273
274     # Líneas horizontales de cotas
275     plt.axhline(I_min[i], color='red', linestyle='--', label='Inventario mínimo')
276     plt.axhline(I_max[i], color='green', linestyle='--', label='Inventario máximo')
277
278     # Líneas verticales donde hubo reabastecimiento
279     for w in refill_weeks:
280         plt.axvline(x=w, color='black', linestyle=':', alpha=0.6)
281
282     plt.title(f"Evolución del inventario - Estación {i+1}")
283     plt.xlabel("Semana")
284     plt.ylabel("Inventario (litros)")
285     plt.grid(True)
286     plt.legend()
287     plt.tight_layout()
288     plt.show()
289
290 import numpy as np
291 import pandas as pd
292 import cvxpy as cp
293 import matplotlib.pyplot as plt
294
295 def main():
296     ruta_excel = r"resultados_estaciones (1).xlsx"
297     df_proyecciones = pd.read_excel(
298         ruta_excel,
299         sheet_name="Proyecciones",
300         index_col=0

```

```

301     )
302
303     n = 5
304     T_total = len(df_proyecciones)
305
306     mu = df_proyecciones.values.T
307
308     sigma = np.zeros_like(mu)
309     for i in range(n):
310         serie = df_proyecciones.iloc[:, i]
311         roll_std = serie.diff().rolling(4).std().fillna(serie.std())
312         sigma[i] = np.maximum(roll_std.values, 1.0)
313
314     np.random.seed(42)
315     eps = np.random.normal(0, sigma)
316     mu_stochastic = np.maximum(mu + eps, 0)
317
318     I_min = np.array([10000, 12000, 9000, 11000, 10000], float)
319     I_max = np.array([68991.05, 55981.45, 49611.06, 24046.678, 51829.93], float)
320     I = np.array([20000]*5, float)
321
322     C = 40000
323     I_hist = []
324     refill_weeks = []
325
326     def optimize_refill(I_current, mu_current):
327         Q = cp.Variable(n, nonneg=True)
328         T = cp.Variable(nonneg=True)
329         constraints = [
330             I_current + Q - mu_current*T >= I_min,
331             I_current + Q <= I_max,
332             cp.sum(Q) <= C
333         ]
334         prob = cp.Problem(cp.Maximize(T), constraints)
335         prob.solve(solver=cp.ECOS)
336         return Q.value, T.value
337
338     for t in range(T_total):
339         I_hist.append(I.copy())
340         d_real = mu_stochastic[:, t]
341         I = I - d_real
342
343         if np.any(I <= I_min):
344             Q_opt, T_opt = optimize_refill(I, mu_stochastic[:, t])
345             refill_weeks.append(t)
346             I = I + Q_opt
347
348     I_hist_arr = np.array(I_hist)
349
350     # Graficas
351     plt.figure(figsize=(15, 7))
352     for i in range(n):
353         plt.plot(I_hist_arr[:, i], label=f"Estación {i+1}", linewidth=2)
354     for w in refill_weeks:
355         plt.axvline(x=w, color='gray', linestyle='--', alpha=0.5)
356     plt.title("Inventarios semanales (todas las estaciones)")
357     plt.xlabel("Semana")
358     plt.ylabel("Inventario (litros)")

```

```

359 plt.grid(True)
360 plt.legend()
361 plt.show()
362
363 for i in range(n):
364     plt.figure(figsize=(12, 5))
365     plt.plot(I_hist_arr[:, i], label=f"Inventario estación {i+1}", linewidth=2)
366     plt.axhline(I_min[i], color='red', linestyle='--', label='Inventario mínimo')
367     plt.axhline(I_max[i], color='green', linestyle='--', label='Inventario máximo')
368     for w in refill_weeks:
369         plt.axvline(x=w, color='black', linestyle=':', alpha=0.6)
370     plt.title(f"Evolución del inventario - Estación {i+1}")
371     plt.xlabel("Semana")
372     plt.ylabel("Inventario (litros)")
373     plt.grid(True)
374     plt.legend()
375     plt.tight_layout()
376     plt.show()
377
378 if __name__ == "__main__":
379     main()

```
